

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
дисциплины «Объектно-ориентированное программирование»
Вариант 7

Выполнила:
Еремина Татьяна Евгеньевна
3 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и
вычислительная техника», очная
форма обучения

(подпись)

Проверил:
Воронкин Р.А., доцент департамента
цифровых, робототехнических систем
и электроники института
перспективной инженерии

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Тема: элементы объектно-ориентированного программирования в языке Python.

Цель: приобретение навыков по работе с классами и объектами при написании программ с помощью языка программирования Python версии 3.x.

Репозиторий: <https://github.com/TL-hash/lab1-OOP>

Практическая часть:

Примеры.

1. Создадим класс Book, он имеет строковую переменную material со значением paper, строковую переменную cover со значением paperback и пустой список в качестве атрибута all_books.

2. Все эти переменные являются атрибутами класса и к ним можно получить доступ, используя точечную запись.

```
1 class Book:
2
3     material = "paper"
4     cover = "paperback"
5     all_books = []
6
7 my_book = Book()
8
9 print(my_book.material)
10 print(my_book.cover)
11 print(my_book.all_books)
```

Рисунок 1 - Создание класса

A screenshot of a Python interpreter window titled "example1". The window shows the output of the code from Figure 1. The output is displayed on four lines: "C:\Users\User", "paper", "paperback", and "[]". On the left side of the window, there are four navigation icons: an up arrow, a down arrow, a refresh/clear icon, and a list/expand icon.

Рисунок 2 - Доступ к атрибутам класса

3. Создадим класс River с конструктором `__init__`.
4. Создадим три экземпляра (или объекта) класса River: `volga`, `seine` и `nile`. Поскольку определены параметры `name` и `length` для `__init__`, они должны быть явно переданы при создании новых экземпляров.

```
1 class River:|
2
3     all_rivers = []
4
5     def __init__(self, name, length):
6         self.name = name
7         self.length = length
8         River.all_rivers.append(self)
9
10    def get_info(self):
11        print("Длина {0} равна {1} км".format(self.name, self.length))
12
13    volga = River("Volga", 10)
14    seine = River("Seine", 10)
15    nile = River("Nile", 10)
16
17    for river in River.all_rivers:
18        print(river.name)
```

Рисунок 3 - Создание класса и его экземпляров

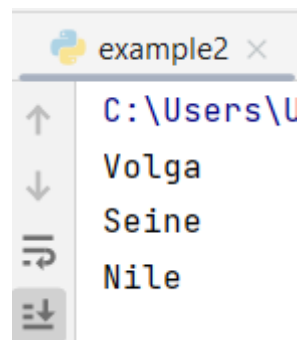


Рисунок 4 - Результат выполнения

5. Добавим метод к нашему классу, чтобы продемонстрировать использование параметра `self`, который используется внутри метода, тогда значения экземпляра сохранятся для последующего использования.

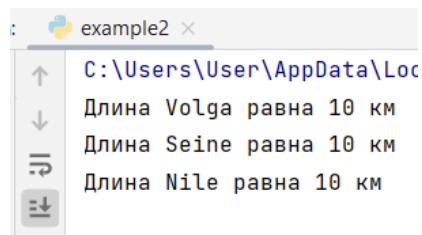


Рисунок 5 - Применение метода

6. Создадим класс Pet. Этот класс имеет три атрибута класса: kind, n_pets и pet_names, а также три атрибута экземпляра, определенных в методе __init__: spec, name и legs.

7. Создадим несколько экземпляров этого класса.

```
1 class Pet:|
2
3     kind = "mammal"
4     n_pets = 0
5     pet_names = []
6
7     def __init__(self, spec, name):
8         self.spec = spec
9         self.name = name
10        self.legs = 4
11
12    tom = Pet("cat", "Tom")
13    avocado = Pet("dog", "Avocado")
14    ben = Pet("goldfish", "Ben")
15
```

Рисунок 6 - Класс Pet и его экземпляры

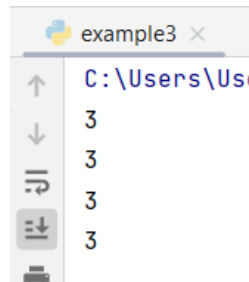
8. Изменим количество питомцев n_pets. Так как n_pets является целым числом, которые относятся к неизменяемому типу данных, можно изменить его значение для всего класса, только если получим к нему доступ непосредственно как к атрибуту класса

```

16 #получим доступ к атрибуту класса напрямую через класс
17 Pet.n_pets += 3
18
19 print(Pet.n_pets)
20 print(tom.n_pets)
21 print(avocado.n_pets)
22 print(ben.n_pets)

```

Рисунок 7 - Доступ к атрибуту напрямую через класс



```

example3 x
C:\Users\Us
3
3
3
3

```

Рисунок 8 - Результат выполнения

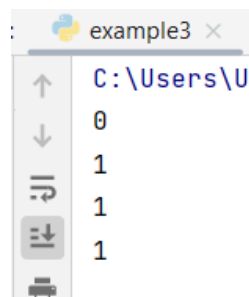
9. Если бы попытались изменить значение `n_pets` через экземпляры, то ничего бы не получилось.

```

24 #доступ к атрибуту класса через экземпляр
25 tom.n_pets += 1
26 avocado.n_pets += 1
27 ben.n_pets += 1
28
29 print(Pet.n_pets)
30 print(tom.n_pets)
31 print(avocado.n_pets)
32 print(ben.n_pets)

```

Рисунок 9 - Доступ к атрибуту класса через экземпляр



```

example3 x
C:\Users\U
0
1
1
1

```

Рисунок 10 - Результат выполнения

10. Несмотря на то, что все экземпляры имеют доступ к атрибуту класса, если эти атрибуты неизменны, изменение их значения для одного экземпляра не изменит их для всего класса. То же самое будет и с атрибутом `kind`.

```
34 #изменение атрибута одного экземпляра не изменит их для всего класса
35     ben.kind = "fish"
36
37     print(Pet.kind)
38     print(tom.kind)
39     print(avocado.kind)
40     print(ben.kind)
```

Рисунок 11 - Неизменный атрибут

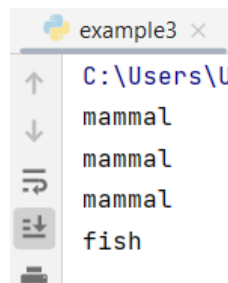


Рисунок 12 - Результат выполнения

11. Атрибут `pet_names` является изменяемым списком, поэтому изменения влияют на весь класс.

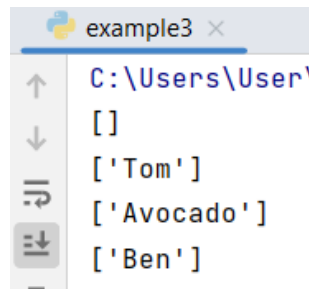
```
42 #атрибут pet_names является изменяемым списком, поэтому изменения
43 # влияют на весь класс
44     tom.pet_names.append(tom.name)
45     avocado.pet_names.append(avocado.name)
46     ben.pet_names.append(ben.name)
47
48     print(Pet.pet_names)
49     print(tom.pet_names)
50     print(avocado.pet_names)
51     print(ben.pet_names)
```

Рисунок 13 - Атрибут изменяемый список

12. Для сохранения атрибутом разных значений для разных экземпляров, можно создать новый список вместо добавления к существующему.

```
53 #создание нового списка для сохранения атрибутом класса pet_names
54 # разных значений для разных экземпляров
55 tom.pet_names = ["Tom"]
56 avocado.pet_names = ["Avocado"]
57 ben.pet_names = ["Ben"]
58
59 print(Pet.pet_names)
60 print(tom.pet_names)
61 print(avocado.pet_names)
62 print(ben.pet_names)
```

Рисунок 14 - Создание нового списка



```
example3 x
C:\Users\User\
[]
['Tom']
['Avocado']
['Ben']
```

Рисунок 15 - Результат выполнения

13. В дополнение к изменению атрибутов также можно создавать атрибуты для класса или конкретного экземпляра.

```
64 #добавление атрибутов
65 Pet.all_specs = [tom.spec, avocado.spec, ben.spec]
66
67 print(tom.all_specs)
68 print(avocado.all_specs)
69 print(ben.all_specs)
```

Рисунок 16 - Добавление атрибутов



Рисунок 17 - Результат выполнения

14. Для понимания различия между методами и функциями (методы – часть структуры ООП, а функции – нет), создадим класс Ship, метод и функцию.

```
1 class Ship:
2
3     def __init__(self, name, capacity):
4         self.name = name
5         self.capacity = capacity
6         self.cargo = 0
7
8     def sail(self):
9         print("{} has sailed!".format(self.name))
10
11     def convert_cargo(self):
12         cargo_kg = self.cargo * 1000
13         return cargo_kg
14
15
16 def sail_function(name):
17     print("{} has sailed!".format(name))
```

Рисунок 18 - Создание класса, метода и функции

15. И метод, и функция дают одинаковые результаты, но по-разному. Метод связан с объектом класса, он не является независимым, а вот функция является. Они оба вызываются по именам, но для вызова метода нужно вызвать также и класс, к которому принадлежит этот метод.

```
19 # #создание экземпляра класса Ship и вызов метода sail
20 black_pearl = Ship("Black Pearl", 800)
21 black_pearl.sail()
22
23 #вызов функции sail_function
24 sail_function(black_pearl.name)
25
26 print(black_pearl.convert_cargo())
```

Run: example4 x

C:\Users\User\AppData\Local\Programs\Python\Python311\p
Black Pearl has sailed!
Black Pearl has sailed!
0

Рисунок 19 - Результат вызова

16. В методе `convert_cargo`, который вычисляет, сколько килограммов груза везет корабль, не изменено дефолтное значение атрибута `cargo`, поэтому метод вернет 0 умноженное на 1000.

17. Атрибуты экземпляра определяются в методах, поэтому по определению можно создавать новые атрибуты внутри пользовательских методов.

18. Для этого создадим класс `Ship`.

```
1 class Ship:
2
3     def __init__(self, name, capacity):
4         self.name = name
5         self.capacity = capacity
6         self.cargo = 0
7
```

Рисунок 20 - Класс и конструктор

19. Введем метод `name_captain`, который создает новый атрибут с именем `captain` и печатает соответствующее сообщение.

```

8      def name_captain(self, cap):
9          self.captain = cap
10         print("{} is the captain of the {}".format(self.name, self.captain))
11

```

Рисунок 21 - Метод name_captain

20. Создадим корабль и вызовем новый метод.

```

29      black_pearl = Ship("Black Pearl", 800)
30
31      # # ошибка, так как атрибут captain создается только в методе name_captain
32      # print(black_pearl.captain) #AttributeError
33
34
35      black_pearl.name_captain("Jack Sparrow")
36
37      print(black_pearl.captain)

```

Рисунок 22 - Вызов метода

21. Можно использовать методы для изменения атрибутов экземпляра, создадим методы load_cargo (проверяет, что загрузка определенного веса не будет превышать грузоподъемность судна) и unload_cargo (проверяет, не сделает ли разгрузка вес груза отрицательным).

```

12      def load_cargo(self, weight):
13          if self.cargo + weight <= self.capacity: #if weight <= self.free.space()
14              self.cargo += weight
15              print("Loaded {} tons".format(weight))
16          else:
17              print("Cannot load that much")
18
19      def unload_cargo(self, weight):
20          if self.cargo - weight >= 0:
21              self.cargo -= weight
22              print("Unloaded {} tons".format(weight))
23          else:
24              print("Cannot unload that much")

```

Рисунок 23 - Методы для изменения атрибутов экземпляров

22. Они оба вносят изменения или выводят сообщение, что требуемые изменения невозможны.

```
#пример
black_pearl.load_cargo(600)
black_pearl.unload_cargo(400)
black_pearl.load_cargo(700)
black_pearl.unload_cargo(300)
```

example5 x

C:\Users\User\AppData\Local\Pr
Loaded 600 tons
Unloaded 400 tons
Cannot load that much
Cannot unload that much

Рисунок 24 - Результат выполнения

23. Рассмотрим класс прямоугольника Rectangle.

```
1 class Rectangle:
2
3     def __init__(self, width, height):
4         self._width = width
5         self._height = height
6
7     def get_width(self):
8         return self._width
9
10    def set_width(self, w):
11        self._width = w
12
13    def get_height(self):
14        return self._height
15
16    def set_height(self, h):
17        self._height = h
18
19    def area(self):
20        return self._width * self._height
```

Рисунок 25 - Класс Rectangle

24. Для доступа к `_width` и `_height` используются специальные методы, но ничего не мешает обратиться к ним напрямую.

```
#прямое обращение к атрибутам
rect = Rectangle(10, 20)
print(rect.get_width())

print(rect._width)
```

example6 x

C:\Users\User\AppData\Local\Pr

10

10

Рисунок 26 - Прямое обращение к атрибутам

25. Если атрибут задан с двух подчеркиваний, напрямую уже не обратиться.

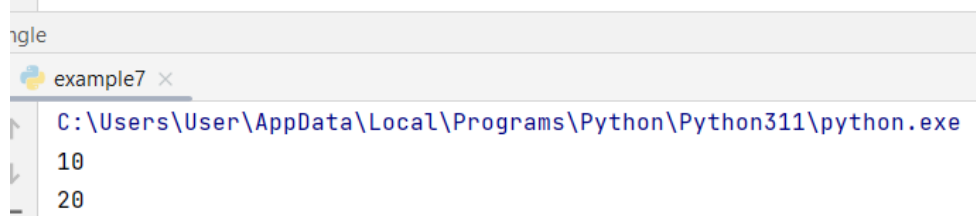
```
1 class Rectangle:
2
3     def __init__(self, width, height):
4         self.__width = width
5         self.__height = height
6
7     def get_width(self):
8         return self.__width
9
10    def set_width(self, w):
11        self.__width = w
12
13    def get_height(self):
14        return self.__height
15
16    def set_height(self, h):
17        self.__height = h
18
19    def area(self):
20        return self.__width * self.__height
21
```

Рисунок 27 - Модифицированный класс

26. С двумя подчеркиваниями нужно работать только через `get_width()` или как `_Rectangle__width`.

```
#с двумя подчеркиваниями нужно работать только через get_width()
rect = Rectangle(10, 20)
print(rect.get_width())

#атрибут теперь носит название _Rectangle__width
rect._Rectangle__width = 20
print(rect.get_width())
```



The screenshot shows a Python IDE window titled 'example7'. The console output displays the execution of the code. The first line of output is '10', and the second line is '20'. The code in the editor defines a Rectangle class with a get_width() method and demonstrates how to access the width attribute using the get_width() method and how to set the width attribute using the _Rectangle__width attribute.

Рисунок 28 - Результат выполнения

27. Свойством называется такой метод класса, работа с которым подобна работе с атрибутом. Для объявления метода свойством необходимо использовать декоратор `@property`.

28. Сделаем реализацию класса `Rectangle` с использованием свойств.

```

1  class Rectangle:
2
3      def __init__(self, width, height):
4          self.__width = width
5          self.__height = height
6
7      @property
8      def width(self):
9          return self.__width
10
11     @width.setter
12     def width(self, w):
13         if w > 0:
14             self.__width = w
15         else:
16             raise ValueError
17
18     @property
19     def height(self):
20         return self.__height
21
22     @height.setter
23     def height(self, h):
24         if h > 0:
25             self.__height = h
26         else:
27             raise ValueError
28
29     def area(self):
30         return self.__width * self.__height

```

Рисунок 29 - Rectangle со свойствами

29. Теперь можно работать с `width` и `height`, как будто они являются атрибутами, можно не только читать, но и задавать новые значения.

```
#можно работать с width и height, как с атрибутами
rect = Rectangle(10, 20)
print(rect.width)
print(rect.height)

#можно задавать новые значения свойствам
rect.width = 50
print(rect.width)
```

gle > area()

example8 x

C:\Users\User\AppData\Local\Programs\Python\Python3

10
20
50

Рисунок 30 - Работа с атрибутами

Пример 1. Рациональная (несократимая) дробь представляется парой целых чисел (a, b), где a – числитель, b – знаменатель. Создать класс Rational для работы с рациональными дробями. Обязательно должны быть реализованы операции:

1. сложения add, $(a, b) + (c, d) = (ad + bc, bd)$;
2. вычитания sub, $(a, b) - (c, d) = (ad - bc, bd)$;
3. умножения mul, $(a, b) \times (c, d) = (ac, bd)$;
4. деления div, $(a, b) / (c, d) = (ad, bc)$;
5. сравнения equal, greater, less.

Должна быть реализована приватная функция сокращения дроби reduce, которая обязательно вызывается при выполнении арифметических операций.

Решение: для избегания деления на ноль используется генерация исключительной ситуации valueError. Встроенная функция isinstance(obj, Cls), используемая при реализации методов арифметических операций и операций отношения, позволяет узнать, что некоторый объект obj является

либо экземпляром класса Cls либо экземпляром одного из потомков класса Cls.

Листинг примера:

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

class Rational:

    def __init__(self, a=0, b=1):
        a = int(a)
        b = int(b)

        if b == 0:
            raise ValueError

        self.__numerator = abs(a)
        self.__denominator = abs(b)

        self.__reduce()

    # Сокращение дроби
    def __reduce(self):
        # Функция для нахождения наибольшего общего делителя
        def gcd(a, b):
            if a == 0:
                return b
            elif b == 0:
                return a
            elif a >= b:
                return gcd(a % b, b)
            else:
                return gcd(a, b % a)
        c = gcd(self.__numerator, self.__denominator)

        self.__numerator //= c
        self.__denominator //= c

    @property
    def numerator(self):
        return self.__numerator

    @property
    def denominator(self):
        return self.__denominator

    # Прочитать значение дроби с клавиатуры. Дробь вводится
    # как a/b
    def read(self, prompt=None):
        line = input() if prompt is None else input(prompt)
        parts = list(map(int, line.split('/', maxsplit=1)))
```

```

    if parts[1] == 0:
        raise ValueError()

    self.__numerator = abs(parts[0])
    self.__denominator = abs(parts[1])

    self.__reduce()

# Вывести дробь на экран
def display(self):
    print(f"{self.__numerator}/{self.__denominator}")

# Сложение обыкновенных дробей
def add(self, rhs):
    if isinstance(rhs, Rational):
        a = self.numerator * rhs.denominator + \
            self.denominator * rhs.numerator
        b = self.denominator * rhs.denominator

        return Rational(a, b)
    else:
        raise ValueError

# Вычитание обыкновенных дробей
def sub(self, rhs):
    if isinstance(rhs, Rational):
        a = self.numerator * rhs.denominator - \
            self.denominator * rhs.numerator
        b = self.denominator * rhs.denominator

        return Rational(a, b)
    else:
        raise ValueError

# Умножение обыкновенных дробей
def mul(self, rhs):
    if isinstance(rhs, Rational):
        a = self.numerator * rhs.numerator
        b = self.denominator * rhs.denominator

        return Rational(a, b)
    else:
        raise ValueError

# Деление обыкновенных дробей
def div(self, rhs):
    if isinstance(rhs, Rational):
        a = self.numerator * rhs.denominator
        b = self.denominator * rhs.numerator

        return Rational(a, b)

```

```

        else:
            raise ValueError

# Отношение обыкновенных дробей
def equals(self, rhs):
    if isinstance(rhs, Rational):
        return (self.numerator == rhs.numerator) and \
            (self.denominator == rhs.denominator)
    else:
        return False

def greater(self, rhs):
    if isinstance(rhs, Rational):
        v1 = self.numerator / self.denominator
        v2 = rhs.numerator / rhs.denominator

        return v1 > v2
    else:
        return False

def less(self, rhs):
    if isinstance(rhs, Rational):
        v1 = self.numerator / self.denominator
        v2 = rhs.numerator / rhs.denominator

        return v1 < v2
    else:
        return False

if __name__ == '__main__':
    r1 = Rational(3, 4)
    r1.display()

    r2 = Rational()
    r2.read("Введите обыкновенную дробь: ")
    r2.display()

    r3 = r2.add(r1)
    r3.display()

    r4 = r2.sub(r1)
    r4.display()

    r5 = r2.mul(r1)
    r5.display()

    r6 = r2.div(r1)
    r6.display()

```

Результат выполнения:

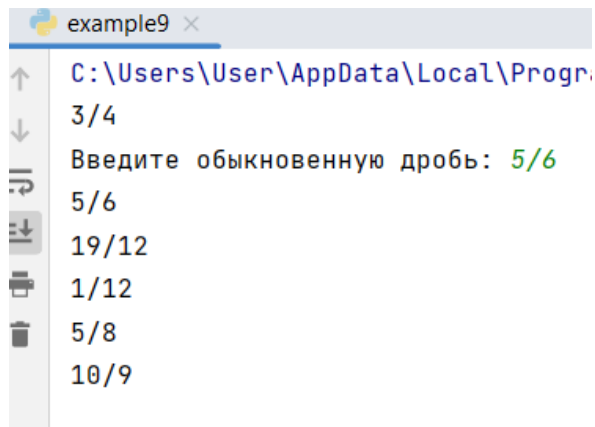


Рисунок 31 - Результат выполнения примера

Задания (вариант 7).

Задание 1. Парой называется класс с двумя полями, которые обычно имеют имена `first` и `second`. Требуется реализовать тип данных с помощью такого класса. Во всех заданиях обязательно должны присутствовать:

1. метод инициализации `__init__`; метод должен контролировать значения аргументов на корректность;
2. ввод с клавиатуры `read`;
3. вывод на экран `display`.

Реализовать внешнюю функцию с именем `make_тип()`, где `тип` – тип реализуемой структуры. Функция должна получать в качестве аргументов значения для полей структуры и возвращать структуру требуемого типа. При передаче ошибочных параметров следует выводить сообщение и заканчивать работу.

В раздел программы, начинающийся после инструкции `if __name__ == '__main__':` добавить код, демонстрирующий возможности разработанного класса.

Поле `first` – дробное число, левая граница диапазона; поле `second` – дробное число, правая граница диапазона. Реализовать метод `rangecheck()` – проверку заданного числа на принадлежность диапазону.

Решение: создаётся класс `Diapason`, описывающий числовой отрезок `[first, second]`. Поля класса объявлены приватными (`self.__first`, `self.__second`) – это обеспечивает инкапсуляцию, доступ к ним осуществляется через свойства `first` и `second`, объявленные декоратором `@property`. Конструктор `__init__` принимает две границы (по умолчанию 0.0 и 0.0) и контролирует их корректность: обе границы должны быть числами (`int` или `float`), а левая граница не должна превышать правую. При нарушении любого из условий генерируется исключение `ValueError`. Метод `read` читает строку с клавиатуры, разбирает её на две части, разделённые пробелом, преобразует каждую в `float` и присваивает полям, предварительно проверяя: строка содержит ровно два числа, обе части являются числами, левая граница не превышает правую. Метод `display` выводит диапазон в формате `[first, second]`. Метод `rangecheck(x)` принимает проверяемое число `x` и возвращает `True`, если $\text{first} \leq x \leq \text{second}$, и `False` в противном случае; если `x` не является числом – генерируется `ValueError`. Внешняя функция `make_diapason(first, second)` контролирует корректность входных параметров и возвращает готовый объект класса `Diapason`.

Листинг задания:

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

class Diapason:

    def __init__(self, first=0.0, second=0.0):
        if not isinstance(first, (int, float)) or \
            not isinstance(second, (int, float)):
            raise ValueError

        if first > second:
            raise ValueError

        self.__first = float(first)
        self.__second = float(second)

    @property
    def first(self):
        return self.__first
```

```

@property
def second(self):
    return self.__second

# Чтение границ с клавиатуры
def read(self, prompt=None):
    line = input() if prompt is None else input(prompt)
    parts = line.split()

    if len(parts) != 2:
        raise ValueError

    try:
        first = float(parts[0])
        second = float(parts[1])
    except ValueError:
        raise ValueError

    if first > second:
        raise ValueError

    self.__first = first
    self.__second = second

# Вывод диапазона
def display(self):
    print(f"[{self.__first}, {self.__second}]")

# Принадлежность диапазону
def rangecheck(self, x):
    if not isinstance(x, (int, float)):
        raise ValueError

    return self.__first <= x <= self.__second

# Внешняя функция
def make_diapason(first, second):
    if not isinstance(first, (int, float)) or \
        not isinstance(second, (int, float)):
        raise ValueError

    if first > second:
        raise ValueError

    return Diapason(first, second)

if __name__ == '__main__':
    d1 = Diapason(1.5, 3.7)
    d1.display()

```

```

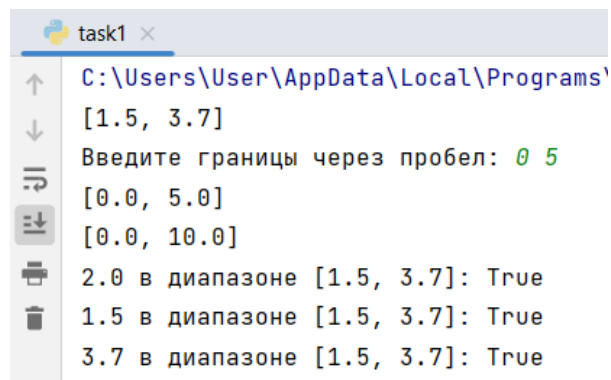
d2 = Diapason()
d2.read("Введите границы через пробел: ")
d2.display()

d3 = make_diapason(0.0, 10.0)
d3.display()

print("2.0 в диапазоне [1.5, 3.7]:", d1.rangecheck(2.0))
print("1.5 в диапазоне [1.5, 3.7]:", d1.rangecheck(1.5))
print("3.7 в диапазоне [1.5, 3.7]:", d1.rangecheck(3.7))

```

Результат выполнения:



```

task1
C:\Users\User\AppData\Local\Programs'
[1.5, 3.7]
Введите границы через пробел: 0 5
[0.0, 5.0]
[0.0, 10.0]
2.0 в диапазоне [1.5, 3.7]: True
1.5 в диапазоне [1.5, 3.7]: True
3.7 в диапазоне [1.5, 3.7]: True

```

Рисунок 32 - Результат выполнения

Задание 2. Составить программу с использованием классов и объектов для решения задачи. Во всех заданиях, помимо указанных в задании операций, обязательно должны быть реализованы следующие методы:

1. метод инициализации `__init__`;
2. ввод с клавиатуры `read`;
3. вывод на экран `display`.

В раздел программы, начинающийся после инструкции `if __name__ == '__main__':` добавить код, демонстрирующий возможности разработанного класса.

Создать класс `Date` для работы с датами в формате «год.месяц.день». Дата представляется структурой с тремя полями типа `unsigned int`: для года, месяца и дня. Класс должен включать не менее трех функций инициализации: числами, строкой вида «год.месяц.день» (например, «2004.08.31») и датой. Обязательными операциями являются: вычисление

даты через заданное количество дней, вычитание заданного количества дней из даты, определение високосности года, присвоение и получение отдельных частей (год, месяц, день), сравнение дат (равно, до, после), вычисление количества дней между датами.

Решение:

1. Создаётся класс `Date`, описывающий календарную дату. Поля класса объявлены приватными (`self.__year`, `self.__month`, `self.__day`) – это обеспечивает инкапсуляцию, доступ к ним осуществляется через свойства `year`, `month` и `day`, объявленные декоратором `@property`.

Конструктор `__init__` поддерживает три способа инициализации:

1) Числами – `Date(2004, 8, 31)`. Передаются три аргумента: год, месяц, день.

2) Строкой – `Date("2004.08.31")`. Если первый аргумент – строка, она разбивается по символу `.` на три части, которые преобразуются в целые числа.

3) Датой – `Date(other_date)`. Если первый аргумент – объект `Date`, значения полей копируются из него.

2. Конструктор контролирует корректность аргументов: год должен быть неотрицательным целым, месяц – от 1 до 12, день – от 1 до количества дней в соответствующем месяце с учётом високосности. При нарушении любого условия генерируется исключение `ValueError`.

3. Метод `__is_leap(year)` определяет, является ли год високосным: год делится на 400 – високосный; делится на 100, но не на 400 – не високосный; делится на 4 – високосный. Метод `__days_in_month(year, month)` возвращает количество дней в месяце: для февраля – 29 в високосный год и 28 в остальные, для остальных месяцев – фиксированные значения из списка.

4. Метод `read` читает строку с клавиатуры в формате год.месяц.день, разбирает её и присваивает полям, предварительно проверяя корректность.

Метод `display` выводит дату в формате ГГГГ.ММ.ДД с ведущими нулями (например, 2004.08.31).

5. Метод `is_leap()` возвращает `True`, если год текущей даты високосный, и `False` в противном случае.

6. Метод `add_days(n)` возвращает новую дату, отстоящую от текущей на `n` дней вперёд. Реализация – цикл на `n` шагов: увеличиваем день на 1; если день превысил количество дней в месяце – сбрасываем его в 1, увеличиваем месяц; если месяц превысил 12 – сбрасываем его в 1, увеличиваем год. Метод `sub_days(n)` работает аналогично, но в обратную сторону: уменьшаем день на 1; если день стал 0 – переходим к предыдущему месяцу и берём его последний день.

7. Метод `days_between(other)` вычисляет количество дней между текущей датой и `other`. Определяется, какая дата меньше (с помощью метода `before`), затем от меньшей даты шагаем вперёд по одному дню, пока не дойдём до большей, подсчитывая шаги.

8. Метод `equals(other)` проверяет равенство двух дат, `before(other)` – что текущая дата раньше `other`, `after(other)` – что позже. Сравнение выполняется поочерёдно: сначала по году, при равенстве – по месяцу, при равенстве – по дню.

9. Методы `set_year`, `set_month`, `set_day` устанавливают соответствующие части даты с проверкой корректности значений.

10. Внешняя функция `make_date(year, month, day)` получает значения полей и возвращает готовый объект класса `Date`.

Листинг задания:

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
```

```
class Date:
```

```
    def __init__(self, year=2000, month=1, day=1):
        # если передали строку – разбираем её
        if isinstance(year, str):
            parts = year.split('.')
            year = parts[0]
            month = parts[1]
            day = parts[2]
```

```

        if len(parts) != 3:
            raise ValueError
        year = int(parts[0])
        month = int(parts[1])
        day = int(parts[2])

    # если передали другую дату – копируем
    if isinstance(year, Date):
        month = year.month
        day = year.day
        year = year.year

    if not isinstance(year, int) or not isinstance(month, int) \
        or not isinstance(day, int):
        raise ValueError

    if year < 0 or month < 1 or month > 12 or day < 1:
        raise ValueError

    if day > self.__days_in_month(year, month):
        raise ValueError

    self.__year = year
    self.__month = month
    self.__day = day

    @property
    def year(self):
        return self.__year

    @property
    def month(self):
        return self.__month

    @property
    def day(self):
        return self.__day

    def __is_leap(self, year):
        if year % 400 == 0:
            return True
        if year % 100 == 0:
            return False
        if year % 4 == 0:
            return True
        return False

    def __days_in_month(self, year, month):
        days = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
        if month == 2 and self.__is_leap(year):
            return 29
        return days[month - 1]

```

```

def read(self, prompt=None):
    line = input() if prompt is None else input(prompt)
    parts = line.split('.')

    if len(parts) != 3:
        raise ValueError

    year = int(parts[0])
    month = int(parts[1])
    day = int(parts[2])

    if year < 0 or month < 1 or month > 12 or day < 1:
        raise ValueError

    if day > self.__days_in_month(year, month):
        raise ValueError

    self.__year = year
    self.__month = month
    self.__day = day

def display(self):
    print(f>{self.__year:04d}.{self.__month:02d}.{self.__day:02d}>>)

def is_leap(self):
    return self.__is_leap(self.__year)

def add_days(self, n):
    if not isinstance(n, int) or n < 0:
        raise ValueError

    year = self.__year
    month = self.__month
    day = self.__day

    for _ in range(n):
        day += 1
        if day > self.__days_in_month(year, month):
            day = 1
            month += 1
            if month > 12:
                month = 1
                year += 1

    return Date(year, month, day)

def sub_days(self, n):
    if not isinstance(n, int) or n < 0:
        raise ValueError

```

```

year = self.__year
month = self.__month
day = self.__day

for _ in range(n):
    day -= 1
    if day < 1:
        month -= 1
        if month < 1:
            month = 12
            year -= 1
        day = self.__days_in_month(year, month)

return Date(year, month, day)

def days_between(self, other):
    if not isinstance(other, Date):
        raise ValueError

    # определяем, какая дата меньше
    if self.before(other):
        small = self
        big = other
    else:
        small = other
        big = self

    # от меньшей даты шагаем вперёд, пока не дойдём до большей
    count = 0
    current = Date(small.year, small.month, small.day)

    while not current.equals(big):
        current = current.add_days(1)
        count += 1

    return count

def equals(self, other):
    if not isinstance(other, Date):
        raise ValueError
    return (self.__year == other.year) and \
           (self.__month == other.month) and \
           (self.__day == other.day)

def before(self, other):
    if not isinstance(other, Date):
        raise ValueError
    if self.__year != other.year:
        return self.__year < other.year
    if self.__month != other.month:
        return self.__month < other.month
    return self.__day < other.day

```

```

def after(self, other):
    if not isinstance(other, Date):
        raise ValueError
    return not self.before(other) and not self.equals(other)

def set_year(self, year):
    if not isinstance(year, int) or year < 0:
        raise ValueError
    self.__year = year

def set_month(self, month):
    if not isinstance(month, int) or month < 1 or month > 12:
        raise ValueError
    self.__month = month

def set_day(self, day):
    if not isinstance(day, int) or day < 1:
        raise ValueError
    if day > self.__days_in_month(self.__year, self.__month):
        raise ValueError
    self.__day = day

def make_date(year=2000, month=1, day=1):
    return Date(year, month, day)

if __name__ == '__main__':
    # Создание числами
    d1 = Date(2004, 8, 31)
    d1.display()

    # Создание строкой
    d2 = Date(«2004.02.29»)
    d2.display()

    # Создание копией
    d3 = Date(d2)
    d3.display()

    # Ввод с клавиатуры
    d4 = Date()
    d4.read(«Введите дату в формате год.месяц.день: «)
    d4.display()

    # Високосность
    print(d1.is_leap())
    print(d2.is_leap())

    # Прибавление / вычитание дней
    d5 = d1.add_days(10)

```

```

d5.display()

d6 = d1.sub_days(10)
d6.display()

# Разница между датами
print(d1.days_between(d2))

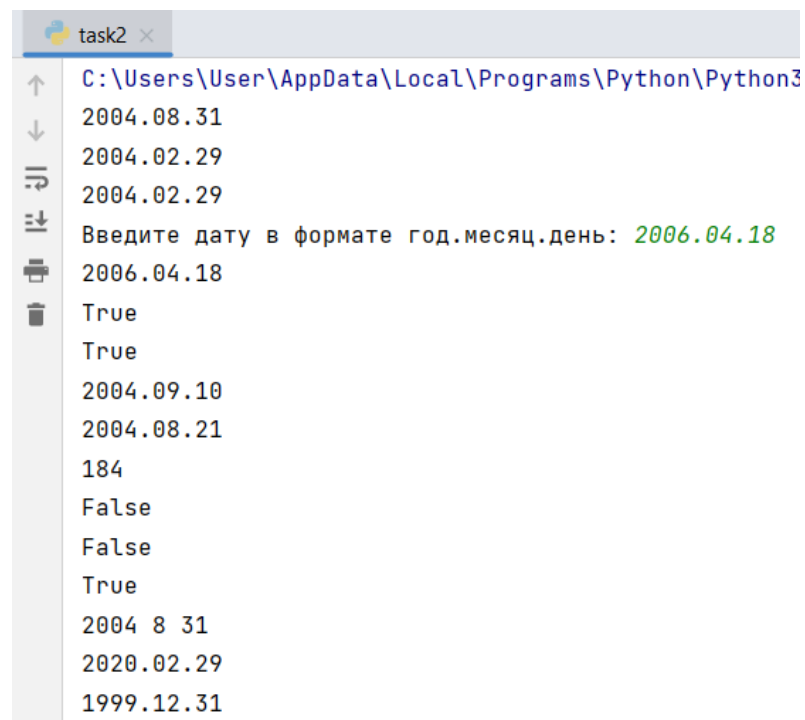
# Сравнения
print(d1.equals(d2))
print(d1.before(d2))
print(d1.after(d2))

# Получение и установка частей
print(d1.year, d1.month, d1.day)
d1.set_year(2020)
d1.set_month(2)
d1.set_day(29)
d1.display()

d7 = make_date(1999, 12, 31)
d7.display()

```

Результат выполнения:



```

task2 x
C:\Users\User\AppData\Local\Programs\Python\Python3
2004.08.31
2004.02.29
2004.02.29
Введите дату в формате год.месяц.день: 2006.04.18
2006.04.18
True
True
2004.09.10
2004.08.21
184
False
False
True
2004 8 31
2020.02.29
1999.12.31

```

Рисунок 33 - Результат выполнения

Вывод: были изучены элементы объектно-ориентированного программирования в языке Python 3.x. В практической части работы были рассмотрены примеры создания классов, их атрибутов (класса и экземпляра), конструктора `__init__`, методов экземпляра, различий между методами и функциями, а также приёмов инкапсуляции данных с использованием одного и двух подчёркиваний. Отдельно рассмотрено применение декоратора `@property` для организации доступа к приватным атрибутам через свойства, что позволяет работать с ними как с обычными полями, но с контролем корректности значений.

Контрольные вопросы:

1. Как осуществляется объявление класса в языке Python?

Класс объявляется с помощью ключевого слова `class`, после которого указывается имя класса (по соглашению – с большой буквы) и двоеточие. Тело класса записывается с отступом. Например: `class Diapason:...`

2. Чем атрибуты класса отличаются от атрибутов экземпляра

Атрибуты класса объявляются непосредственно в теле класса, вне методов, и являются общими для всех экземпляров. Атрибуты экземпляра создаются внутри метода `__init__` (или других методов) через `self.имя` и принадлежат конкретному объекту. Изменение неизменяемого атрибута класса через экземпляр не влияет на класс, а изменение изменяемого атрибута (например, списка) через один экземпляр затрагивает все экземпляры.

3. Каково назначение методов класса?

Методы класса описывают поведение объектов: они определяют действия, которые можно выполнять над объектом или с его помощью. Методы имеют доступ к атрибутам объекта через параметр `self` и позволяют читать и изменять их значения, а также выполнять вычисления, связанные с объектом.

4. Для чего предназначен метод `__init__()` класса?

Метод `__init__()` является конструктором класса. Он автоматически вызывается при создании нового объекта и предназначен для инициализации его атрибутов: присвоения начальных значений полям, проверки корректности переданных аргументов, выполнения подготовительных действий.

5. Каково назначение `self`?

`self` – это ссылка на текущий экземпляр класса, передаваемая первым параметром во все методы экземпляра. Через `self` метод обращается к атрибутам и другим методам именно того объекта, для которого он был

вызван. Python подставляет нужный объект автоматически при вызове метода.

6. Как добавить атрибуты в класс?

Атрибуты класса добавляются непосредственно в теле класса: `class Diapason: count = 0.`

Атрибуты экземпляра добавляются в методе `__init__` (или любом другом методе) через `self`:

```
def __init__(self, first, second):  
    self.first = first  
    self.second = second
```

Также атрибут можно создать для конкретного объекта извне: `объект.имя_атрибута = значение.`

7. Как осуществляется управление доступом к методам и атрибутам в языке Python?

В Python управление доступом реализуется соглашением об именовании. Одно подчёркивание в начале имени (`_name`) означает, что атрибут или метод предназначен для внутреннего использования (защищённый). Два подчёркивания (`__name`) запускают механизм `name mangling` – имя преобразуется в `_Класс__name`, что делает прямой доступ извне затруднительным (приватный атрибут). Доступ к таким атрибутам организуется через свойства `@property` или специальные методы `get_.../set_...`

8. Каково назначение функции `isinstance`?

Функция `isinstance(объект, класс)` проверяет, является ли объект экземпляром указанного класса (или одного из его потомков). Возвращает `True` или `False`. Используется для контроля типов аргументов в методах и для безопасной работы с объектами разных классов. Например: `if isinstance(rhs, Rational):...`